Deep Reinforcement Learning for Connect-*K*: A Comparative Study of Rainbow DQN and Proximal Policy Optimization

Sreekar Patti
Khoury College of Computer Sciences
Northeastern University
Boston, MA 02115
patti.sr@northeastern.edu

Krish Bansal
Khoury College of Computer Sciences
Northeastern University
Boston, MA 02115
bansal.kr@northeastern.edu

Abstract

We present a systematic comparison of two deep reinforcement learning approaches, Rainbow DQN (value-based, off-policy) and Proximal Policy Optimization (PPO, policy-based, on-policy), applied to Connect-*K*, a parametric generalization of Connect Four played on a 6×7 board where aligning k consecutive pieces in any direction wins the game. Both agents share a convolutional neural network encoder and are trained from scratch using a two-phase curriculum: an initial random-opponent phase followed by frozen self-play. We evaluate agents at three difficulty settings ($k \in \{4, 5, 6\}$) against a uniform-random baseline and a depth-3 minimax search, as well as in direct head-to-head competition. On standard Connect-4 (k=4), Rainbow DQN achieves **96%** win rate against random play while PPO reaches **72%**. In head-to-head play, DQN wins **100%** of games against PPO across all k values. Critically, *neither agent wins a single game against the depth-3 minimax baseline*, exposing a fundamental gap between pattern-matching neural policies and even shallow lookahead search. PPO also exhibits a sharp training collapse near the end of its run, which we attribute to self-play instability. Our results highlight the strength of off-policy sample reuse for adversarial board games, while also pointing to the limitations of model-free RL without explicit lookahead.

1 Introduction

Board games have long served as canonical benchmarks for artificial intelligence, from early work on game-tree search and minimax analysis [Knuth and Moore, 1975] to AlphaGo's landmark defeat of world champions [Silver et al., 2016]. Their fully observable, deterministic dynamics make them attractive test-beds: difficulty can be tuned precisely, ground-truth optimal play is sometimes known, and progress is easy to measure.

Connect-*K* is the class of two-player zero-sum games played on a 6×7 board with gravity-obeying column drops, where the first player to form k consecutive pieces (horizontally, vertically, or diagonally) wins. The special case k=4 is the classic *Connect Four*, which was fully solved by Tromp [1995] and Allis [1988]: with optimal play, the first player always wins. By varying k, we obtain a family of games with identical state spaces but tunable strategic depth: larger k demands longer-term planning and makes random play less exploitable, providing a natural difficulty axis for studying agent robustness.

Training a reinforcement learning (RL) agent to play Connect-k presents several non-trivial challenges beyond single-agent settings:

1. **Alternating play.** The reward signal for one player depends on the subsequent action of the opponent, making credit assignment harder than in single-agent environments.
2. **Board-perspective consistency.** A unified policy must reason about board positions from the current player's point of view, regardless of which side it is actually controlling.
3. **Non-stationary opponent.** In self-play, the target policy changes as the agent improves, violating the stationary-MDP assumption.
4. **Sparse, delayed rewards.** All non-terminal transitions yield zero reward; the terminal reward (±1) is received only at the end of a game that can last up to 42 moves.

We address these challenges with (i) a board-flipping convention that always places the current player's pieces in channel 0 of the state tensor, (ii) a two-phase curriculum from random-opponent play to periodic frozen self-play, and (iii) two complementary RL algorithms whose distinct properties illuminate different aspects of the learning problem.

Rainbow DQN [Hessel et al., 2018] is an off-policy, value-based method that combines six orthogonal improvements over vanilla DQN: dueling networks, double Q-learning, prioritized experience replay (PER), multi-step returns, distributional RL, and noisy networks. In our implementation we use all components except distributional RL and noisy nets, retaining four key improvements. Off-policy learning allows sample reuse through a large replay buffer, and PER focuses

computation on high-error, informative transitions.

Proximal Policy Optimization (PPO) [Schulman et al., 2017] is an on-policy, policy-gradient method that optimizes a clipped surrogate objective to prevent excessively large policy updates. Combined with Generalized Advantage Estimation (GAE) [Schulman et al., 2016], it achieves strong empirical performance across continuous and discrete domains. Its on-policy nature means fresh samples are always used, avoiding potential issues with stale replay data.

Our main contributions are:

- A clean, reproducible implementation of both algorithms on a shared Connect-k environment with curriculum self-play.
- A systematic empirical comparison on three difficulty levels ($k \in \{4, 5, 6\}$), including evaluations against a random baseline, a minimax search, and each other.
- An analysis of training dynamics that reveals PPO's instability under self-play and the failure of both model-free agents against a shallow lookahead baseline.

2 Background

2.1 Markov Decision Processes

A Markov Decision Process (MDP) is a tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $T : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ the transition kernel, $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ the reward function, and $\gamma \in [0, 1)$ the discount factor. The agent's goal is to learn a policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ that maximises the expected discounted return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$.

2.2 Connect-k as a Two-Player MDP

We model the game as a two-player zero-sum MDP from the perspective of a single *learning agent* (Player 1). The state $s_t \in \mathbb{R}^{3\times6\times7}$ is a three-channel binary tensor described in Section 4.1. The action space $\mathcal{A} = \{0, \ldots, 6\}$ indexes the seven columns. Illegal moves (dropping into a full column) are penalised with $r = -1$ and terminate the episode. A win yields $r = +1$, a loss from the opponent's winning move yields $r = -1$ for the agent, and a draw yields $r = 0$.

Two-player play is reduced to a single-agent MDP by treating the opponent's action as part of the environment transition: after the agent's move, the opponent selects its action and the next state returned is the board *after* the opponent has played, ready for the agent's next decision.

2.3 Deep Q-Networks (DQN)

Q-learning [Watkins and Dayan, 1992] estimates the optimal action-value function $Q^*(s, a) = \mathbb{E}\big[r + \gamma \max_{a'} Q^*(s', a') \;\big|\; s, a\big]$ from experience. Mnih et al. [2015] showed that parameterising Q with a deep neural network Q_θ and updating it via stochastic gradient descent on the Bellman residual enables human-level play on Atari games. A fixed *target network* Q_{θ^-}, periodically copied from Q_θ, stabilises training. An experience *replay buffer* [Lin, 1992] breaks temporal correlations by training on randomly sampled past transitions.

2.4 Policy Gradient Methods

Policy gradient methods directly optimise the expected return with respect to policy parameters θ using the policy gradient theorem [Sutton et al., 2000]:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s)\, A(s, a)]\,, \qquad (1)$$

where $A(s, a) = Q(s, a) - V(s)$ is the advantage. Actor-Critic methods [Mnih et al., 2016] reduce gradient variance by using a learned value function V_θ to compute advantages. PPO [Schulman et al., 2017] further stabilises updates by clipping the probability ratio between the new and old policies.

3 Related Work

Classical Connect Four solvers. Allis [1988] solved Connect-4 using a knowledge-based approach built on nine hand-crafted strategic rules, without relying on brute-force search. Tromp [1995] later produced a complete solution via an exhaustive 8-ply opening database. These methods achieve provably optimal play but do not generalise to other values of k without re-solving from scratch.

TD-Gammon and tabular self-play. Tesauro [1994] demonstrated that TD(λ) with a neural network trained via self-play could reach expert-level backgammon play, with the strongest version additionally relying on hand-crafted input features. This pioneering result directly motivates our self-play curriculum, though our approach uses modern deep RL algorithms rather than hand-crafted features.

AlphaGo and AlphaZero. AlphaGo [Silver et al., 2016] combined deep convolutional networks with Monte Carlo Tree Search (MCTS) and supervised pretraining to defeat world champions at Go. AlphaZero [Silver et al., 2018] subsequently showed that self-play alone, without any human priors, was sufficient across Go, chess, and shogi. MCTS provides a principled, lookahead-based search that dramatically improves policy quality at inference time. We deliberately exclude MCTS to isolate the learning algorithms themselves and to maintain tractable training times on commodity hardware.

Neural Fictitious Self-Play. Heinrich and Silver [2016] proposed a framework for imperfect-information games where each agent trains a best-response policy against a mixture of past policies. Our frozen self-play is a simplified variant: we maintain a single frozen snapshot of the current policy rather than a mixture, which is sufficient for the perfect-information Connect-k domain.

Comparison of DQN and PPO. Several works compare value-based and policy-gradient algorithms on Atari [Hessel et al., 2018, Mnih et al., 2016] and continuous control benchmarks, generally finding complementary strengths. To

our knowledge, no prior work provides a controlled comparison specifically on parametric board games with a shared self-play curriculum, which is the gap this paper addresses.

4 Project Description

4.1 State Representation

The raw 6×7 board is encoded as a three-channel tensor $s \in \{0,1\}^{3\times 6\times 7}$:

- **Channel 0:** Binary map of the *current player's* pieces.
- **Channel 1:** Binary map of the *opponent's* pieces.
- **Channel 2:** Binary valid-move mask: 1 where column c is not full (equivalently, board$[0, c] = 0$).

Crucially, the channel assignment is always relative to the *active player*, not to an absolute player identity. Whenever the board state is passed to the opponent's policy, channels 0 and 1 are swapped. This *board-flipping* convention ensures that a single learned policy can be deployed for either side without modification, and that the Q-function or value function always has a consistent semantic meaning.

4.2 Neural Network Architecture

Both agents share an identical convolutional **encoder**:

1. Conv2d$(3 \to 64, 3\times 3, \text{pad} = 1)$ + ReLU
2. Conv2d$(64 \to 128, 3\times 3, \text{pad} = 1)$ + ReLU
3. Conv2d$(128 \to 128, 3\times 3, \text{pad} = 1)$ + ReLU
4. Flatten $\to$ Linear$(128 \times 6 \times 7, 512)$ + ReLU

The output is a 512-dimensional feature vector for any input board state. Padding is set to 1 to preserve spatial dimensions across all three convolutional layers, allowing the network to reason about piece patterns near the board edges.

Dueling DQN head (for Rainbow DQN): two independent streams applied to the 512-d features:

$$V(s) = W_V^{(2)}\,\sigma(W_V^{(1)} f) \in \mathbb{R}, \tag{2}$$

$$A(s,a) = W_A^{(2)}\,\sigma(W_A^{(1)} f) \in \mathbb{R}^7, \tag{3}$$

where σ denotes ReLU, each hidden layer has 256 units. The Q-values are assembled as:

$$Q(s,a;\theta) = V(s) + \left(A(s,a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s,a') \right). \tag{4}$$

Subtracting the mean advantage removes a degree of freedom that would otherwise make the decomposition non-unique [Wang et al., 2016].

Actor-Critic head (for PPO): two independent streams applied to the same 512-d features:

$$\text{logits}(s) = W_\pi^{(2)}\,\sigma(W_\pi^{(1)} f) \in \mathbb{R}^7, \tag{5}$$

$$V_\phi(s) = W_v^{(2)}\,\sigma(W_v^{(1)} f) \in \mathbb{R}, \tag{6}$$

each with a hidden dimension of 256. The policy is defined as $\pi_\theta(a \mid s) \propto \exp(\text{logits}(s)_a + m_a)$, where $m_a = -10^9$ for invalid columns ($a \notin$ valid) and 0 otherwise. Both architectures have approximately 480 K parameters.

4.3 Rainbow DQN

Our Rainbow DQN agent [Hessel et al., 2018] combines four key improvements over vanilla DQN.

4.3.1 Multi-Step Returns.

Rather than bootstrapping from the immediate next state, we accumulate the n-step return:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q_{\theta^-}(s_{t+n}, a^*), \tag{7}$$

with $n = 3$. This reduces the variance of the target at the cost of slightly increased bias, and speeds propagation of non-zero rewards through long trajectories.

4.3.2 Double Q-Learning.

The Double DQN target [van Hasselt et al., 2016] decouples action *selection* (online network Q_θ) from action *evaluation* (target network Q_{θ^-}):

$$a^* = \arg\max_{a'} Q_\theta(s_{t+n}, a'), \quad y = G_t^{(n)} + \gamma^n Q_{\theta^-}(s_{t+n}, a^*). \tag{8}$$

This corrects the maximisation bias inherent in vanilla DQN.

4.3.3 Prioritized Experience Replay.

Following Schaul et al. [2016], transitions are stored in a binary sum-tree with priority

$$p_i = (|\delta_i| + \epsilon_{\text{PER}})^\alpha, \tag{9}$$

where δ_i is the TD error and $\epsilon_{\text{PER}} = 10^{-6}$ prevents zero priority. Sampling probability is proportional to $p_i^\alpha / \sum_j p_j^\alpha$ with $\alpha = 0.6$. To correct the resulting distributional bias, each transition i is weighted by the importance-sampling (IS) coefficient

$$w_i = \frac{(N \cdot P(i))^{-\beta}}{\max_j w_j}, \tag{10}$$

with β annealed from $0.4 \to 1.0$ over 5 000 update steps. Stratified sampling ensures that each priority segment is represented in every batch.

4.3.4 Dueling Networks.

Equation (4) constitutes the fourth improvement. The dueling head allows the value stream to learn $V(s)$ without requiring a specific action to be sampled, which is particularly beneficial in Connect-k where many actions lead to equally good states.

Training objective. The Huber loss weighted by IS coefficients is minimised:

$$\mathcal{L} = \mathbb{E}_i\left[w_i\, \ell_\delta\big(y_i - Q_\theta(s_i, a_i)\big) \right], \tag{11}$$

where ℓ_δ is the Huber function with $\delta = 1$. Gradients are clipped to norm ≤ 10. The target network is hard-updated every 500 gradient steps.

Algorithm 1 Rainbow DQN Training

Initialise online net Q_θ, target net $Q_{\theta^-} \leftarrow Q_\theta$
Initialise PER buffer $\mathcal{B}$, $\varepsilon \leftarrow 1.0$, step $\leftarrow 0$
for episode $= 1, \ldots, 5000$ **do**
  if episode $= 500$ **then** set opponent to frozen self-play
  end if
  if using self-play $\wedge$ episode $\bmod 250 = 0$ **then** re-freeze opponent
  end if
  $s \leftarrow$ RESET()
  repeat
    $a \leftarrow \varepsilon$-greedy$(Q_\theta, s, \text{valid})$
    $r, s', \text{done} \leftarrow$ STEP(a); push $(s, a, r, s', \text{done})$ to n-step buffer
    if n-step buffer full $\vee$ done **then**: flush n-step transition to $\mathcal{B}$
    end if
    if $|\mathcal{B}| \geq$ batch_size **then**
      Sample $(s_i, a_i, G_i, s'_i, d_i, w_i) \sim \mathcal{B}$; compute target via Eq. (8)
      Update θ by gradient descent on IS-weighted Huber loss
      Update priorities in $\mathcal{B}$; increment step
      if step $\bmod 500 = 0$ **then**: $Q_{\theta^-} \leftarrow Q_\theta$
      end if
    end if
    $s \leftarrow s'$; $\varepsilon \leftarrow \max(0.05,\ \varepsilon \cdot 0.9995)$
  until done
end for

4.4 Proximal Policy Optimization (PPO)

4.4.1 Generalized Advantage Estimation.

After collecting a rollout of $T = 128$ environment steps, we compute advantages via GAE [Schulman et al., 2016]:

$$\delta_t = r_t + \gamma\, V_\phi(s_{t+1})(1 - d_t) - V_\phi(s_t), \tag{12}$$

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l\, \delta_{t+l}, \tag{13}$$

with $\gamma = 0.99$ and $\lambda = 0.95$. The returns are $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$. Advantages are normalised to zero mean and unit variance before computing the policy loss:

$$\tilde{A}_t = \frac{\hat{A}_t - \mu_{\hat{A}}}{\sigma_{\hat{A}} + 10^{-8}}. \tag{14}$$

4.4.2 Clipped Surrogate Objective.

The probability ratio $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ is clipped to $[1 - \varepsilon,\ 1 + \varepsilon]$ with $\varepsilon = 0.2$:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t \tilde{A}_t,\ \text{clip}(r_t, 1{-}\varepsilon, 1{+}\varepsilon)\, \tilde{A}_t\big)\Big]. \tag{15}$$

Algorithm 2 PPO Training

Initialise network (π_θ, V_ϕ), buffer $\mathcal{D}$ of size $T{=}128$
total_steps $\leftarrow 0$
while total_steps $< 500\,000$ **do**
  if total_steps $= 50\,000$ **then** set opponent to frozen self-play
  end if
  if using self-play $\wedge$ total_steps $\bmod 25\,000 = 0$ **then** re-freeze
  end if
  Collect T steps into $\mathcal{D}$; total_steps $+= T$
  Compute GAE advantages via Eq. (13); set $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$
  Normalise advantages: $\tilde{A} \leftarrow (\hat{A} - \mu)/(\sigma + 10^{-8})$
  for epoch $= 1, \ldots, 4$ **do**
    for each minibatch $\mathcal{M} \subset \mathcal{D}$ (size 32) **do**
      Compute $r_t(\theta)$; evaluate clipped loss Eq. (15)
      Update θ on loss Eq. (16); clip $\|\nabla\| \leq 0.5$
    end for
  end for
  Reset $\mathcal{D}$
end while

4.4.3 Combined Objective.

The full loss combines the clipped policy objective, a value function loss, and an entropy bonus to encourage exploration:

$$L(\theta) = -L^{\text{CLIP}}(\theta) + c_1 L^{\text{VF}}(\theta) - c_2\, \mathcal{H}[\pi_\theta(\cdot \mid s_t)], \tag{16}$$

with $c_1 = 0.5$ (value coefficient) and $c_2 = 0.01$ (entropy coefficient). L^{VF} is the mean-squared error between predicted values and GAE returns. After each rollout, we run $K = 4$ epochs of minibatch gradient descent (batch size 32) with gradient clipping at $\|\nabla\| \leq 0.5$.

4.5 Self-Play Curriculum

Both agents begin training against a *random opponent* that selects uniformly among valid columns. This warm-up phase allows the agent to discover basic tactics (blocking open columns, creating threats) without the instability of self-play against an untrained policy.

After a fixed threshold (episode 500 for DQN, which is 10% of total training, and step 50 000 for PPO), the opponent is replaced with a *frozen snapshot* of the current policy network. The frozen weights are copied periodically (every 250 episodes / 25 000 steps) rather than continuously, preventing distribution collapse: if the opponent updates simultaneously with the agent, both can co-adapt in a cycle that never improves absolute board strength. The frozen opponent retains $\varepsilon_{\text{opp}} = 0.05$ random exploration to avoid degenerate deterministic cycles.

5 Experiments

5.1 Experimental Setup

All experiments were conducted on a single machine with an NVIDIA GPU. Both algorithms are implemented in PyTorch [Paszke et al., 2019] with automatic mixed precision disabled for reproducibility, and both agents use the Adam optimizer [Kingma and Ba, 2015]. Each agent is trained independently for each value of k; there is no weight sharing across k settings.

Table 1 lists all hyperparameters. The values were selected based on published recommendations for each algorithm [Hessel et al., 2018, Schulman et al., 2017] with minor adjustments for the Connect-k domain (smaller batch and replay buffer than the Atari benchmarks, reflecting the shorter episodes and simpler state space).

Table 1: Hyperparameters for both agents.

Hyperparameter	DQN	PPO
Optimiser	Adam	Adam
Learning rate	1×10^{-4}	3×10^{-4}
Discount γ	0.99	0.99
Batch size	64	32
DQN-specific		
Replay capacity	100 000	–
Target update (steps)	500	–
n-step	3	–
PER α	0.6	–
PER β start	0.4	–
ε start/end	1.0 / 0.05	–
ε decay	0.9995/ep	–
Grad clip norm	10.0	–
PPO-specific		
Rollout length T	–	128
GAE λ	–	0.95
Clip ε	–	0.2
Value coef c_1	–	0.5
Entropy coef c_2	–	0.01
Epochs per update	–	4
Grad clip norm	–	0.5
Curriculum (both)		
Self-play start	ep 500	step 50 000
Opponent re-freeze	every 250 ep	every 25 000 steps
Opp exploration ε	0.05	0.05
Training budget	5 000 ep	500 000 steps
Eval frequency	every 250 ep	every 10 000 steps

Evaluation protocol. During training, win rates are measured every evaluation checkpoint over 100 games against the *current training opponent* (random or frozen self-play). Final evaluation uses 100 games against the random baseline and 100 games against the minimax search (depth 3 with alpha-beta pruning). Head-to-head evaluation uses 200 games (100 per starting player) to control for first-mover advantage.

Minimax baseline. Our minimax agent uses alpha-beta pruning [Knuth and Moore, 1975] with a simple terminal-only heuristic: positions that are won or lost score ± 1; all other leaf nodes score 0 (no hand-crafted board evaluation). At depth 3, the agent can detect immediate threats and forced wins but lacks any positional reasoning beyond that horizon.

5.2 Learning Curves

Figure 1 shows training dynamics for both agents on Connect-4 (k=4).

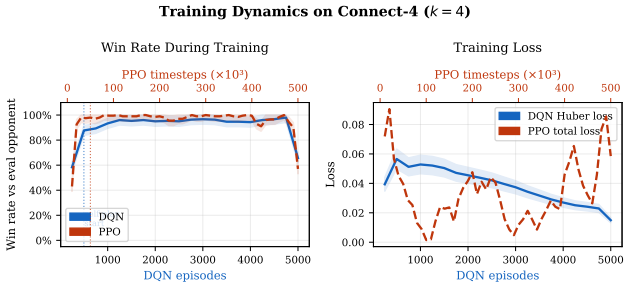

Figure 1: Training dynamics on Connect-4. **Left**: win rate against the current training opponent (random before the vertical dotted line; frozen self-play after). **Right**: Huber loss (DQN) and total PPO loss. Shaded bands indicate ± 4 percentage points.

DQN dynamics. DQN learns quickly: by episode 250 it already achieves 91% win rate against the random opponent, reflecting rapid value learning enabled by the replay buffer. After switching to self-play at episode 500, the win rate dips briefly before recovering and stabilising in the 92–99% range for the rest of training. The Huber loss decreases monotonically from 0.067 at episode 250 down to 0.021 at episode 5 000, indicating consistently improving Q-value estimates.

PPO dynamics. PPO also converges quickly during the random phase, reaching 86% by timestep 10 000 and 99% by timestep 20 000. However, PPO's training is significantly less stable than DQN's. The most striking event is a catastrophic collapse at the end of training: the win rate drops from $\sim$97% to just 23% at the final checkpoint (timestep 500 000). This occurs after a fresh self-play opponent freeze, and the on-policy update rule is unable to quickly re-adapt. Similar but less severe instabilities appear in the k=5 and k=6 runs (Figure 4). This is a practical limitation of on-policy methods when the self-play opponent changes abruptly.

Sample efficiency. DQN's 5 000 episodes correspond to roughly 90 000 environment steps (approximately 18 moves per game on average), versus PPO's 500 000 timesteps. Despite using 5.5$\times$ fewer environment interactions, DQN maintains competitive win rates throughout training, demonstrating the value of off-policy replay for sample efficiency.

5.3 Final Evaluation Results

Table 2 and Figure 2 summarise the full evaluation for k=4.

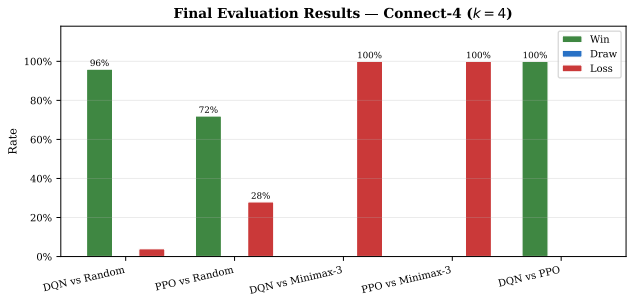

Figure 2: Final evaluation results for Connect-4 (k=4) over 100 games vs. random, 100 games vs. Minimax-3, and 200 head-to-head games. Neither agent wins against Minimax-3.

Table 2: Final evaluation results (Win / Draw / Loss rates), 100 games per matchup except DQN vs PPO (200 games).

	$k=4$			$k=5$			$k=6$		
Matchup	W	D	L	W	D	L	W	D	L
DQN vs Rand	.96	.00	.04	.97	.02	.01	.75	.24	.01
PPO vs Rand	.72	.00	.28	.83	.07	.10	.48	.43	.09
DQN vs MM-3	.00	.00	1.0	.00	.00	1.0	.00	.00	1.0
PPO vs MM-3	.00	.00	1.0	.00	.00	1.0	.00	1.0	.00
DQN vs PPO	1.0	.00	.00	1.0	.00	.00	1.0	.00	.00

Against the random baseline. Both agents decisively beat the random opponent. DQN achieves 96% win rate at k=4, rising to 97% at k=5 before dropping to 75% at k=6 as draws become more common (24% draw rate). PPO reaches 72% at k=4 and 83% at k=5, but drops to 48% wins and 43% draws at k=6. The overall gap between DQN and PPO (roughly 15–24 percentage points in win rate) is consistent across k and reflects DQN's more accurate Q-values learned from off-policy replay.

Against Minimax-3. Neither agent wins a single game against the depth-3 minimax at k=4 or k=5. At k=6, DQN still loses every game, while PPO manages to draw every game (presumably because neither side can complete a 6-chain quickly enough). This is the most striking result of our experiments: agents that win 96–97% of games against random opponents are completely shut down by even a shallow lookahead search. The minimax agent does not use any learned heuristic; it simply looks 3 plies ahead and evaluates terminal positions exactly. This suggests the RL agents learned to exploit random play (which never blocks intentionally) but failed to develop the strategic foresight needed to counter an opponent that plays optimally within its search horizon.

Head-to-head. DQN wins all 200 head-to-head games against PPO across every value of k. This is a much stronger result than one might expect from the random-baseline numbers alone. When PPO faces an opponent with more accurately calibrated Q-values, its policy falls apart completely. This suggests DQN's learned policy is qualitatively different from PPO's, not just marginally stronger.

5.4 Performance Across Difficulty Levels

Figure 3 shows how win and draw rates against the random opponent scale across $k \in \{4, 5, 6\}$. Figure 4 shows the training curves for k=5 and k=6.

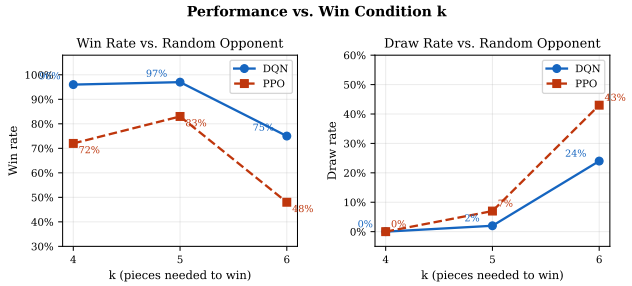

Figure 3: Left: win rate vs. random opponent across k. **Right**: draw rate vs. random opponent across k, showing the large increase in draws at k=6.

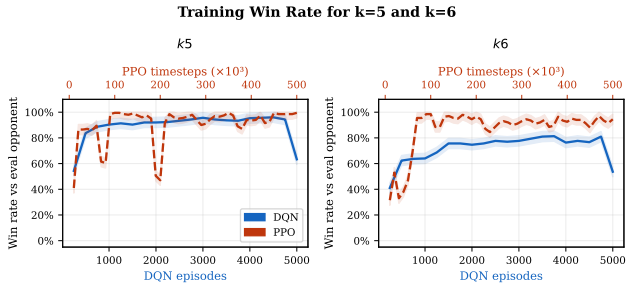

Figure 4: Training win rate for k=5 (left) and k=6 (right). PPO shows repeated instabilities and crashes, particularly at k=5.

Several observations stand out from the cross-k results:

(i) Win rates drop with k, but draws increase sharply. DQN's win rate falls from 96% at k=4 to 75% at k=6, while PPO's drops from 72% to 48%. The decline is partly offset by draws: at k=6, DQN draws 24% of games against random and PPO draws 43%. This makes sense; when k is large, completing a winning chain is harder and both players are more likely to fill the board without a winner.

(ii) PPO instability worsens with k. As shown in Figure 4, PPO's win-rate curves at k=5 and k=6 exhibit multiple large drops, sometimes falling to near 0% before recovering. DQN's curves are comparatively smooth across all three k values. The off-policy replay buffer appears to stabilise DQN by retaining transitions from earlier, better-behaved policies even when the current self-play opponent is strong.

(iii) The minimax failure is universal. Across all k values, neither DQN nor PPO wins a single game against minimax-3 (Table 2). The failure is not a k-specific phenomenon; it reflects a fundamental inability of the model-free policies to

handle an opponent with explicit lookahead, regardless of the difficulty setting.

5.5 Analysis and Discussion

Why DQN outperforms PPO. We attribute DQN's consistent edge to three factors:

1. **Sample reuse.** The replay buffer holds 100 000 transitions; each gradient step draws from the entire training history. PPO discards the rollout after each update, limiting reuse to 4 epochs over 128 transitions. When the self-play opponent shifts abruptly, DQN can still learn from past experience while PPO is stuck with only the most recent (now misleading) rollout.
2. **Prioritised sampling.** PER focuses updates on high-TD-error transitions, which tend to be the tactically critical board positions where the agent's estimates are most wrong. This accelerates learning of the key patterns that distinguish good play.
3. **n-step bootstrapping.** The 3-step return propagates the win/loss signal further back in a single update, which is important in Connect-k where the decisive sequence often spans several moves.

Why neither agent beats minimax. The total failure against depth-3 minimax deserves direct discussion. Our minimax does not use a heuristic evaluation function; it only knows about terminal positions. Yet it still wins every game. We believe the reason is that the RL agents learned a policy calibrated for opponents that do not actively block threats (i.e., random play and frozen self-play snapshots that were not themselves optimal blockers). A depth-3 search, however, will always detect an immediate threat or 3-move forced win and respond to it. The RL agents appear to rely on multi-move setups that go unblocked against random opponents but are trivially deflected by even shallow search. This points to a well-known limitation of model-free RL in deterministic, perfect-information games: without some form of lookahead at inference time, the policy struggles against opponents that do think ahead [Silver et al., 2016, 2018].

PPO's practical limitations. Beyond the performance gap, PPO's training instability is a real practical concern. The catastrophic win-rate collapse near the end of the $k{=}4$ run (from $\sim$97% to 23%) and repeated crashes in $k{=}5$ mean that an engineer deploying PPO for self-play board games would need to implement careful checkpointing and rollback logic. DQN does not exhibit this behavior. That said, PPO has genuine advantages: it requires no replay buffer (saving $\approx$300 MB of memory), its entropy bonus provides automatic exploration without a tuned decay schedule, and it is simpler to implement correctly.

Limitations. Both agents are evaluated as the first-mover only; a more rigorous study would train and test as each player separately. Training was done for a single run per $(k, \text{algorithm})$ combination, so we cannot report confidence intervals. Finally, a stronger minimax baseline (e.g., depth 7 with a heuristic evaluation function) or an MCTS agent would reveal whether the failure is specific to depth-3 search or a more fundamental limitation.

6 Conclusion

We presented a controlled comparison of Rainbow DQN and PPO on Connect-k, a parametric two-player board game. Both algorithms learn competitive policies from scratch using a shared convolutional encoder and a two-phase self-play curriculum. The results are clear: DQN dominates PPO in head-to-head play (100% win rate across all k values) and achieves substantially higher win rates against random opponents (96% vs 72% at $k{=}4$). DQN's advantage comes from off-policy sample reuse, prioritized experience replay, and multi-step bootstrapping, which together produce more accurate Q-values and more stable training.

The most important finding, however, is that *neither agent wins a single game against a depth-3 minimax search*. This reveals a fundamental gap between pattern-matching neural policies trained on random and self-play opponents and even shallow tree-based lookahead. Both agents learned to exploit opponents that do not actively block threats, but failed to generalize to an opponent that does.

Future work. The most natural next step is adding Monte Carlo Tree Search at inference time, as in AlphaGo Zero [Silver et al., 2017] and AlphaZero [Silver et al., 2018], which would give the policy access to the lookahead it currently lacks. Other promising directions include distributional RL (C51 or QR-DQN) for better value uncertainty estimates, training against a stronger minimax baseline to force genuine strategic learning, and a league-training framework to prevent catastrophic forgetting during self-play.

References

Victor Allis. A knowledge-based approach of Connect-Four. *ICCA Journal*, 11(4):165–166, 1988. Master's thesis, Vrije Universiteit, 1988.

Johannes Heinrich and David Silver. Deep reinforcement learning from self-play in imperfect-information games. In *NIPS Deep Reinforcement Learning Workshop*, 2016. arXiv:1603.01121.

Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, pages 3215–3222, 2018.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations*, 2015. arXiv:1412.6980.

Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975. doi: 10.1016/0004-3702(75)90019-3.

Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992. doi: 10.1007/BF00992699.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.

Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1928–1937. PMLR, 2016.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035, 2019.

Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *Proceedings of the 4th International Conference on Learning Representations*, 2016. arXiv:1511.05952.

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *Proceedings of the 4th International Conference on Learning Representations*, 2016. arXiv:1506.02438.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587): 484–489, 2016. doi: 10.1038/nature16961.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550 (7676):354–359, 2017. doi: 10.1038/nature24270.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419): 1140–1144, 2018. doi: 10.1126/science.aar6404.

Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems 12*, pages 1057–1063. MIT Press, 2000.

Gerald Tesauro. TD-Gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation*, 6(2):215–219, 1994. doi: 10.1162/neco.1994.6.2.215.

John Tromp. Connect-4 data set. https://tromp.github.io/c4/c4.html, 1995.

Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, pages 2094–2100, 2016.

Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1995–2003. PMLR, 2016.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992. doi: 10.1007/BF00992698.